

实验五：ROM IP 核和视频播放

实验日期：2025.11.21

姓名：何宗霖

学号：23374275

班级：231516

任课教师：李家军

一、实验目的

1. 掌握 IP 核的使用方法。
2. 了解 LED 点阵的显示原理，培养系统化设计思维。

二、实验原理

1. ROM IP 核原理

ROM（只读存储器）IP 核是一种用于存储固定数据的集成电路，其数据在制造时被预先写入，无法在正常运行时进行修改。

对于数据存储，ROM 内部存储单元通常由多个存储单元组成，每个存储单元可以存储一个比特的信息。数据在制造过程中被烧录，存储在存储单元中。

对于地址解码，ROM 使用地址输入信号选择特定的存储单元。地址线的数量决定了 ROM 的大小。例如：地址线为 n 时，可以访问 2^n 个存储单元。

对于数据输出，读取数据时，根据输入的地址，ROM 将相应的存储单元内容输出到数据总线上。输出通常是组合逻辑，与输入的地址信号直接相关。

1. ROM IP核原理

The image shows the MegaWizard Plug-In Manager interface for the ROM: 1-PORT IP core. On the left, a block diagram of the ROM core is shown with labels: '地址' (Address) pointing to 'address[7..0]', '输出数据' (Output Data) pointing to 'q[7..0]', and '时钟' (Clock) pointing to 'clock'. The core is labeled 'rom_8x256' and '8 bits 256 words'. On the right, the MegaWizard configuration window is shown. It has tabs for 'Parameter Settings', 'EDA', and 'ROM: 1-PORT'. The 'ROM: 1-PORT' tab is active, showing configuration options for the 'rom_8x256' block. A red arrow points from the text '数据宽度，一个地址输出多少位数据' (Data width, how many bits of data are output per address) to the 'How wide should the 'q' output bus be?' field, which is set to 8 bits. Another red arrow points from the text '数据深度，表示一共有多少个地址 256=2^8，所以address宽度为8' (Data depth, representing how many addresses there are in total, 256=2^8, so the address width is 8) to the 'How many 8-bit words of memory?' field, which is set to 256 words. The 'Currently selected device family' is 'Cyclone IV GX'. The 'Match project/default' checkbox is checked. The 'What should the memory block type be?' section has 'Auto' selected. The 'What docking method would you like to use?' section has 'Single clock' selected. The 'Resource Usage' section shows '1 M9K'.

【注】：

数据宽度：表示一个地址输出多少位数据；

数据深度：表示一共有多少个地址。

2..mif 文件原理

.mif 文件就是存储器初始化文件，即 memoryinitializationfile，用来配置 RAM 或 ROM 中的数据。

.mif 文件每 16 个 16bits 的数据（如地址 0-15、地址 16-31）存储了视频一帧的点阵数据，每帧的第一个地址（如 0、16）为点阵的第一列，第二个地址的数据为第二列，以此类推。

可以将.mif 文件复制到项目路径下以便在创建 ROM 核时引用。

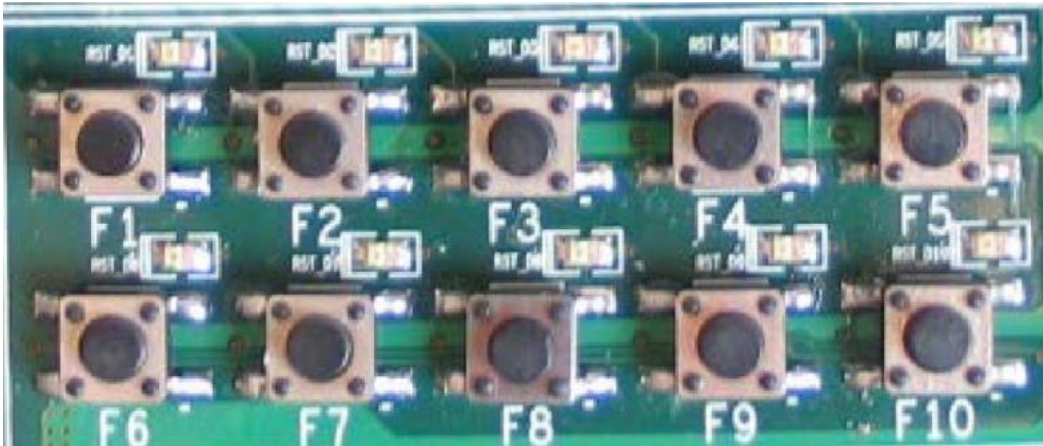
三、实验内容：16*16 点阵视频播放器

- 1. 通过 16*16 点阵屏实现二值视频播放，视频长度不少于 5 秒，帧率不低于 30fps。
- 2. 播放器实现暂停、快进、快退、切换视频功能。

四、硬件资源

设备仪器：FPGA 芯片（EP4CE10F17C8N）

【外设：点动开关】

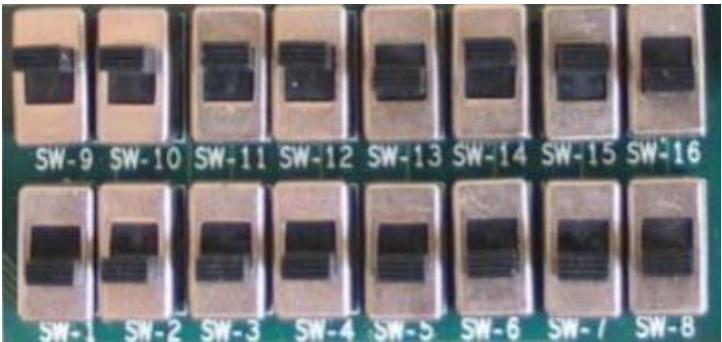


KSI 拨下时 F7-F10 可用

电动开关未按下时输出高电平，按下时输出低电平。

F1			CON1. 10	AB14
F2			CON1. 11	W20
F3			CON1. 12	AA15
F4			CON1. 13	AB15
F5			CON1. 14	T15
F6			CON1. 15	U20
F7	SMBUS_SDA	由开关 KSI 选择	CON1. 16	AA18
F8	SMBUS_SCL	由开关 KSI 选择	CON1. 17	AA16
F9	I2C_SCL	由开关 KSI 选择	CON1. 18	AB18
F10	I2C_SDA	由开关 KSI 选择	CON1. 19	AB19

【外设：拨码开关】

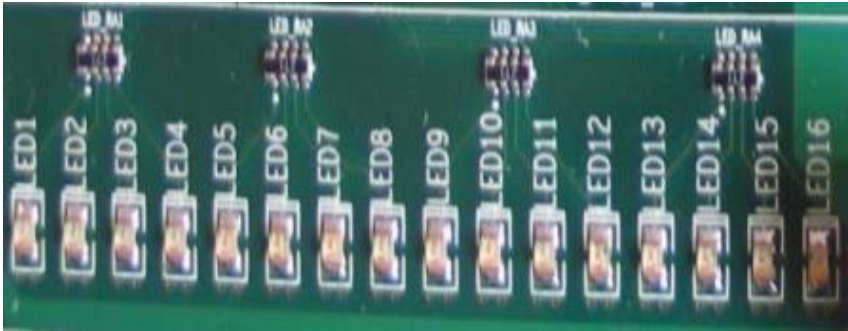


开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。

第一组：SW1，SW2，SW3，SW4，SW5，SW6，SW7，SW8 已经固定连接到实验平台中的 FPGA_CON1 和 FPGA_CON2 处；第二组：SW9，SW10，SW11，SW12，SW13，SW14，SW15，SW16 如果使用该模块，请 把控制拨码开关模块 CTRL_SW 中开关 SEL1，SEL2 拨置于下逻辑电平为 00，使 DP9 数码管显示 1。

M16	SW1
R19	SW2
AA14	SW3
AA1	SW4
A3	SW5
B4	SW6
C4	SW7
E6	SW8
V16	SW9
U14	SW10
B3	SW11
E5	SW12
A4	SW13
B5	SW14
A5	SW15
A6	SW16

【外设：LED 指示灯】



开发平台上有 2×8 共 16 个 LED 指示灯，便于对用户的操作给出提示，使用户得到相应的反馈信息，也可验证设计的正确性，低电平时亮。

第一组：LED1，LED2，LED3，LED4，LED5，LED6，LED7，LED8。如果使用该模块，请把控制拨码开关模块 LCD_ALONE_CTRL_SW 中开关 VLPO 拨置于下为低电平来确定。

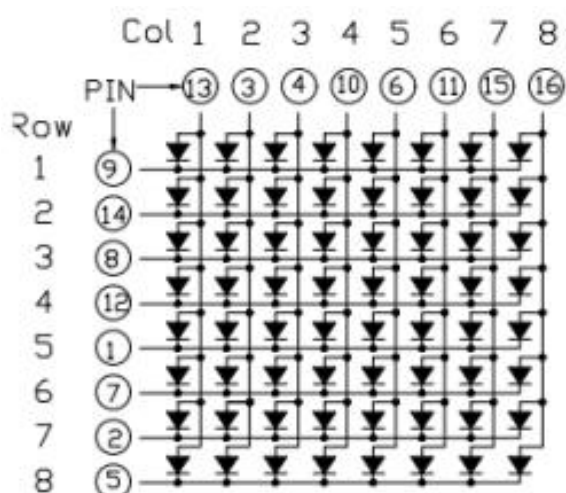
第二组：LED9，LED10，LED11，LED12，LED13，LED14，LED15，LED16。如果使用该模块，请把控制拨码开关 CTRL1_SW 中 SEL1 拨置于上，SEL2 拨置于下，逻辑电平为 10，使 DP9 数码管显示 2。

LED1	OV_D5	由开关 VLPO 选择	CON1. 2	C10
LED2	OV_D6	由开关 VLPO 选择	CON1. 3	E9
LED3	OV_D7	由开关 VLPO 选择	CON1. 4	D10
LED4	OV_D8	由开关 VLPO 选择	CON1. 5	V13
LED5	OV_D9	由开关 VLPO 选择	CON1. 6	V14
LED6	VSYN	由开关 VLPO 选择	CON1. 7	Y17
LED7	PCLK	由开关 VLPO 选择	CON1. 8	W17
LED8	OV_RES	由开关 VLPO 选择	CON1. 9	W19

【外设：8*8 点阵】

8*8 点阵扫描控制电路的光点是从显示模块左上角像素点为起始点扫描，终止于右下角像素点。本实验系统所用的点阵是行共阴、列共阳。

当列共阳为高电平，行共阴为低电平，则所接的发光像素点点亮，反之则所接的发光像素处于截止状态不发光。16*16 LED 点阵由 4 个 8*8 的 LED 阵级联而成。



点阵屏驱动：16 个行选，4 个列选。行选不一定为独热码，列选为 BCD 编码，建议一次更新一列。需合理地选择 ROM 的位宽以与扫描方式适配。

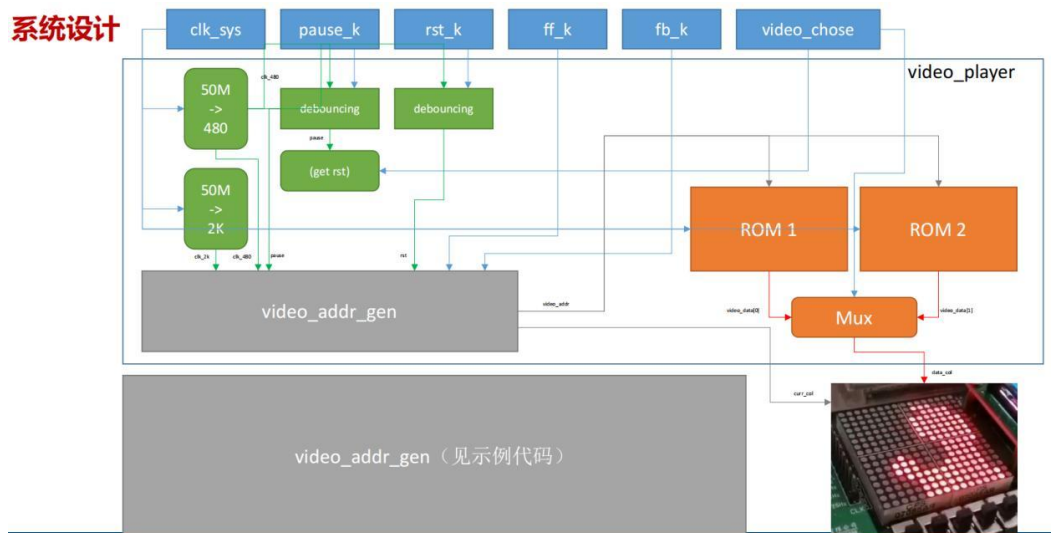
rom 位宽为 16，可以将这 16 位显示在一列上， 每一位对应一个 led 灯。和动态扫描类似，只有一列是亮的，col[3:0]=3'd3 则表示第 3 列亮，即 rom 读出的数据显示在第 3 列。

row[4]	D19	B13	ROW12
row[3]	E19	B14	ROW13
row[2]	F18	A16	ROW14
row[1]	F20	B16	ROW15
row[0]	G20	B17	ROW16

col[3]	F12	C3	COL4
col[2]	K16	C15	COL3
col[1]	J19	F13	COL2
col[0]	H20	C13	COL1
row[15]	F14	B7	ROW1
row[14]	C16	C6	ROW2
row[13]	H15	A8	ROW3
row[12]	A5	C7	ROW4
row[11]	B6	C8	ROW5
row[10]	B7	F8	ROW6
row[9]	A9	F10	ROW7
row[8]	A12	F11	ROW8
row[7]	D17	E12	ROW9
row[6]	C18	B10	ROW10
row[5]	G15	A10	ROW11

五、实验过程与结果

1. 画出系统框图



2. 重叠检测。

实验核心代码：

第一部分：**debouncing** 模块（内容同之前两次实验，不过多赘述）

本模块一共有两个 **always** 模块，其中一个负责计时，另一个负责控制 **key** 信号的稳定。

每当时钟 **clk_sys** 的下降沿触发时，计数器 **count** 增加。如果 **count** 达到 **delayT**，并且当前按键未被按下 ($\sim key_b$)，且 **key** 是高电平（表示消抖完成），则将 **count** 复位为 0。

当计数器 **count** 达到 **delayT-1**，并且按键被按下时 ($\sim key_b$)，并且标志位 **already_up** 为高（表示按键之前已经是松开的），则将 **key** 设置为 0（表示按键已经按下），并且更新 **already_up** 为 0。

如果 **key_b** 为高（按键松开），则 **already_up** 被置为 1，表示准备检测下一次按下。如果 **key** 为低（按键已经按下），则将其重新设置为高（表示按键松开）。

第二部分：video_addr_gen 模块

```
1 module video_addr_gen (clk_480, clk_2K, pause, ff_k, fb_k, rst, video_addr, curr_col);
2
3     input wire clk_480, clk_2K, pause, ff_k, fb_k, rst; // 快进ff_k和快退fb_k不去抖
4     output wire [14:0] video_addr; // 输出到ROM的地址video_addr, 单个视频ROM地址范围: 0-32767 (仅使用了0-28799)
5     output reg [3:0] curr_col = 4'b0000;
6
7     // 使用pause键切换playing的状态
8     reg playing = 1'b1;
9     always @(negedge pause) playing = ~playing; // 异步暂停
10
11     reg [14:0] frame_addr = 15'd0; // frame_addr是当前帧的起始地址
12     always @(negedge clk_480) begin
13         // 若检测到rst或frame_addr到达视频末地址28799, 归零frame_addr
14         if ((~rst)|(frame_addr==15'd28800))begin
15             frame_addr <= 15'd0;
16             // curr_col <= 4'b0000; //补充上的, 理论上得有(同一个curr_col不能被两个always驱动)
17         end
18         // 若本帧刷新完毕, 根据当前{playing,ff_k,fb_k}的不同状态, 选择对帧地址的操作。
19         // 提示: 对帧地址的每次操作不是+1或-1
20         else begin
21             if (curr_col==4'b1111) begin
22                 case ({playing,ff_k,fb_k})//这三个都是低电平有效
23                     3'b011 : frame_addr <= frame_addr + 16; //正常播放
24                     3'b001 : frame_addr <= frame_addr + 32; //在播放的状态下直接快进
25                     3'b010 : frame_addr <= frame_addr - 32; //在播放的状态下直接快退
26                     //000不存在, 默认—暂停
27                     3'b101 : frame_addr <= frame_addr + 32; //不播放也能快进
28                     3'b110 : frame_addr <= frame_addr - 32; //不播放也能快退
29                     default: frame_addr <= frame_addr; //默认是暂停
30                 endcase
31             end
32         end
33     end
34 end
35 // // 使用clk_2K对列选curr_col进行递增, 如果使用clk_480会出现抖动
36 // always @(negedge clk_2K)begin
37 //     curr_col <= curr_col + 1;
38 // end//用于实现视觉滞留
39 always @(negedge clk_2K or negedge rst) begin
40     if (~rst) begin
41         curr_col <= 4'b0000;
42     end else begin
43         curr_col <= curr_col + 1;
44     end
45 end
46 // 输出到ROM的地址实际为帧地址frame_addr和列选(相对地址)curr_col之和
47 assign video_addr = frame_addr + curr_col;
48
49 endmodule
```

该模块主要实现的功能是根据不同的输入控制信号生成视频地址, 供 ROM 读取视频帧数据。同时还实现了通过按键输入来控制视频的播放、快进、快退, 并且使用了不同的时钟信号来控制帧的更新与列选择。

快进、快退、暂停的关键步骤

```
if (curr_col==4'b1111) begin
    case ({playing,ff_k,fb_k})//这三个都是低电平有效
        3'b011 : frame_addr <= frame_addr + 16; //正常播放
        3'b001 : frame_addr <= frame_addr + 32; //在播放的状态下直接快进
        3'b010 : frame_addr <= frame_addr - 32; //在播放的状态下直接快退
        //000 不存在, 默认—暂停
        3'b101 : frame_addr <= frame_addr + 32; //不播放也能快进
```

```

3'b110 : frame_addr <= frame_addr - 32; //不播放也能快退
default: frame_addr <= frame_addr; //默认是暂停
endcase
end

```

这里默认为暂停状态，也可设置为其他。

此外，该模块有一个细节需要【注意】：

```

always @(negedge clk_2K or negedge rst) begin
    if (~rst) begin
        curr_col <= 4'b0000;
    end else begin
        curr_col <= curr_col + 1;
    end
end

```

在 rst 到来的时候，应该同时将 curr_col 清零，但如果直接写会发生报错，因为整个模块中就会出现两处 alway@() 对 curr_col 赋值。因此，为了避免这样的报错发生，我将对 curr_col 的赋值作为整体，写在同一个 alway@() 下，而不是以功能（清零）作为整体去写。

第三部分：video_player 模块

```

1 module video_player (clk_sys, pause_k, ff_k, fb_k, rst_k, video_choose, data_col, curr_col);
2
3     input wire clk_sys, pause_k, ff_k, fb_k, rst_k;
4     input wire [1:0] video_choose;
5     output reg [15:0] data_col;
6     output wire [3:0] curr_col;
7
8     // 480hz分频模块，480=30fps*16col,这是说每秒钟要显示30帧，每帧16列，所以每秒钟要刷新480次
9     reg clk_480 = 1'b0;
10    reg [15:0] clk_480_count = 16'd0;
11    always @(negedge clk_sys) begin
12        if (clk_480_count<16'd52083) clk_480_count <= clk_480_count+1;
13        else begin clk_480_count <= 16'b0;
14            clk_480 <= ~clk_480;
15        end
16    end
17
18    // 2Khz分频模块
19    reg clk_2K = 1'b0;
20    reg [15:0] clk_2K_count = 16'd0;
21    always @(negedge clk_sys) begin
22        if (clk_2K_count<16'd12500) clk_2K_count <= clk_2K_count+1;
23        else begin clk_2K_count <= 16'b0;
24            clk_2K <= ~clk_2K;
25        end
26    end
27
28    wire pause, rst_manual;
29    debouncing #(4,6) debouncing_obj1(clk_480, pause_k, pause);
30    debouncing #(4,6) debouncing_obj2(clk_480, rst_k, rst_manual);

```


改进代码，使用 2025 年的代码（逻辑更加清晰）：

```
1 module video_addr_gen (clk_sys, pause, ff_k, fb_k, rst, video_addr, curr_col);
2
3     input wire clk_sys, pause, ff_k, fb_k, rst; // 快进ff_k和快退fb_k不去抖
4     output wire [14:0] video_addr; // 输出到ROM的地址video_addr, 单个视频ROM地址范围: 0-32767 (仅使用了0-28799)
5     output [3:0] curr_col;
6
7
8     // 使用pause键切换playing的状态
9     reg playing;
10
11     //使用计数器进行粗略分频
12     reg [21:0] div_cnt;
13
14     // 计数
15     always @(posedge clk_sys or negedge rst) begin
16         if(~rst) begin
17             div_cnt <= 0;
18         end else begin
19             div_cnt <= div_cnt+1;
20         end
21     end
22
23     //实现暂停功能
24     always @(negedge rst or negedge pause) begin
25         if(~rst) begin
26             playing <= 1;
27         end
28         else if(~pause) begin
29             playing <= ~playing;
30         end
31     end
32
33     //实现16列扫描使能, 约2KHz频率对列选curr_col进行递增
34     assign curr_col = div_cnt[17:14]; //2KHz的变化频率, 与25MHz相差约10K, 变化慢10K
35
36     //帧地址
37     reg [14:0] frame_addr;
38
39     reg prev_cnt_bit;
40
41     //快进、快退、正常播放功能 (确定frame_addr的状态机)
42     always @(negedge clk_sys or negedge rst) begin
43         if(~rst) begin
44             frame_addr <= 15'd0;
45             prev_cnt_bit <= 0;
46         end
47         else begin
48             prev_cnt_bit <= div_cnt[20];
49             if(frame_addr >= 15'd28800) begin
50                 frame_addr <= 15'd0;
51             end
52             else begin
53                 //按照每秒30帧的频率去更新frame_addr, 25MHz是30Hz的约1M倍, 即2的20次方
54                 //即div_cnt的第多少位变化的适合, 更新frame_addr呢?
55                 //使用prev_cnt_bit来定位div_cnt某一位的变化
56                 if(div_cnt[20] != prev_cnt_bit) begin
57                     case ({playing, ff_k, fb_k})
58                         3'b011 : frame_addr <= frame_addr + 16; //正常播放
59                         3'b001 : frame_addr <= frame_addr + 32; //在播放的状态下直接快进
60                         3'b010 : frame_addr <= frame_addr - 32; //在播放的状态下直接快退
61                         //000不存在, 默认—暂停
62                         3'b101 : frame_addr <= frame_addr + 32; //不播放也能快进
63                         3'b110 : frame_addr <= frame_addr - 32; //不播放也能快退
64                         default: frame_addr <= frame_addr; //默认是暂停
65                     endcase
66                 end
67             end
68         end
69     end
70
71     // 输出到ROM的地址实际为帧地址frame_addr和列选 (相对地址) curr_col之和
72     assign video_addr = frame_addr + curr_col;
73 endmodule
```

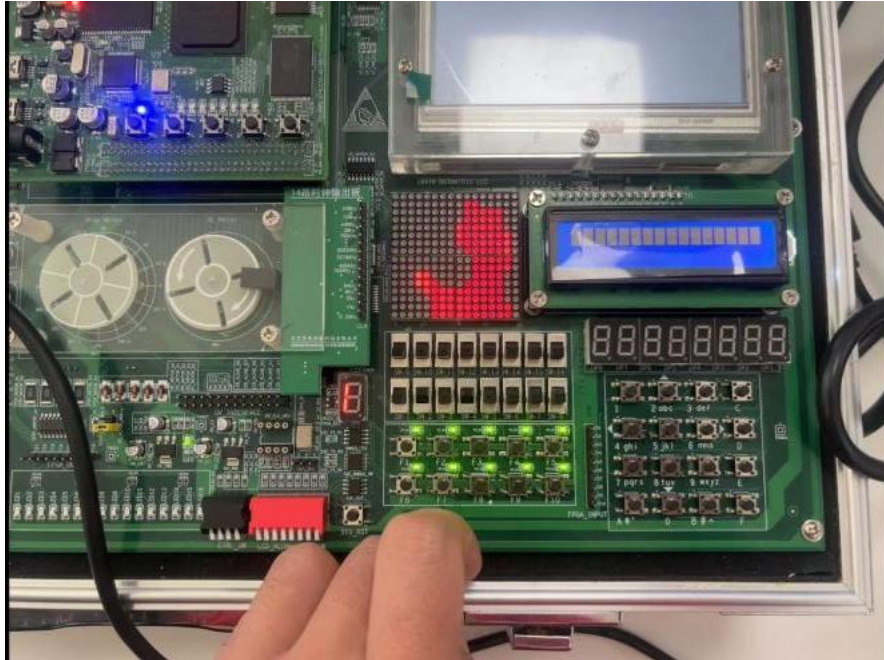
```

1  module video_player (
2      clk_sys, pause_k, ff_k, fb_k, rst_k, video_choose, data_col, curr_col
3  );
4
5      input wire clk_sys, pause_k, ff_k, fb_k, rst_k;
6      input wire [1:0] video_choose;
7      output reg [15:0] data_col;
8      output wire [3:0] curr_col;
9
10
11     wire pause, rst_manual;
12     debouncing debouncing_obj1(.clk_sys(clk_sys), .key_b(pause_k), .key(pause), .key_pulse());
13     debouncing debouncing_obj2(.clk_sys(clk_sys), .key_b(rst_k), .key(rst_manual), .key_pulse());
14
15     // 视频切换时需要重置
16     reg rst_switch;
17     reg [1:0] video_choose_d; // 延时寄存器, 用于检测变化
18     //reg [15:0] rst_delay_count = 16'd0; // 延时计数器
19
20     always @(posedge clk_sys) begin
21         // 检测 video_choose 是否变化
22         // 补充代码
23         if (video_choose != video_choose_d) begin
24             rst_switch <= 1'b0; // 触发重置
25         end else begin
26             rst_switch <= 1'b1; // 清除重置信号
27         end
28         video_choose_d <= video_choose; // 更新视频选择的状态
29     end
30
31     wire rst;
32     assign rst = rst_switch & rst_manual;
33
34     wire [14:0] video_addr;
35     video_addr_gen video_addr_gen_obj(clk_sys, pause, ff_k, fb_k, rst, video_addr, curr_col);
36
37     wire [15:0] video_data[1:0];
38     romV_bad_apple ba(.address(video_addr), .clock(clk_sys), .q(video_data[0]));
39     romV_chicken_you_are_so_beautiful jntm(.address(video_addr), .clock(clk_sys), .q(video_data[1]));
40
41
42     always @(*) begin
43         case (video_choose)
44             2'b10: data_col = video_data[0];
45             2'b11: data_col = video_data[1];
46             default: data_col = 16'b1111_0000_1111_0000;
47         endcase
48     end
49
50 endmodule

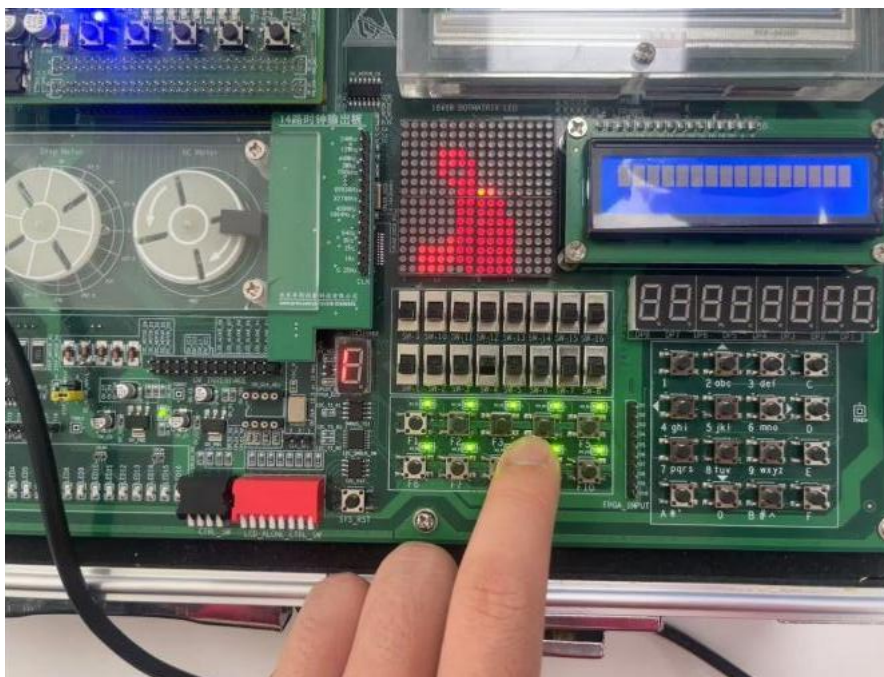
```

最终结果：

视频 1: bad apple



视频 2: 坤坤



实验结果表明通过 16*16 点阵屏实现二值视频播放，实验结果已录制视频并放置于文件夹里，证明播放器实现暂停、快进、快退、切换视频功能。

六、实验仿真波形

1. 重叠检测

编写 video_player_tb.v 代码

```
1  `timescale 1ns / 1ps
2
3  // 测试视频播放器模块
4  module video_player_tb;
5
6      // 定义测试模块的输入输出
7      reg clk_sys_tb;
8      reg pause_k_tb;
9      reg ff_k_tb;
10     reg fb_k_tb;
11     reg rst_k_tb;          // 注意：低电平有效（对应模块内部 rst_n）
12     reg [1:0] video_choose_tb;
13     wire [15:0] data_col_tb;
14     wire [3:0] curr_col_tb;
15
16     // 实例化被测模块
17     video_player_for_test uut (
18         .clk_sys(clk_sys_tb),
19         .pause_k(pause_k_tb),
20         .ff_k(ff_k_tb),
21         .fb_k(fb_k_tb),
22         .rst_k(rst_k_tb),    // 连接到低有效复位
23         .video_choose(video_choose_tb),
24         .data_col(data_col_tb),
25         .curr_col(curr_col_tb)
26     );
27     // 生成系统时钟：周期 10ns (100 MHz)
28     initial begin
29         clk_sys_tb = 0;
30         forever #5 clk_sys_tb = ~clk_sys_tb;
31     end
32     // 监控信号变化（可选，用于终端打印）
33     initial begin
34         $monitor("Time=%0t | curr_col=%0d | pause=%b | ff=%b | fb=%b | video_sel=%b",
35             $time, curr_col_tb, pause_k_tb, ff_k_tb, fb_k_tb, video_choose_tb);
36     end
37
38     // 主测试流程
39     initial begin
40         // 设置 VCD 波形文件
41         $dumpfile("video_player_for_test.vcd");
42         $dumpvars(1, video_player_tb);
43         // 初始化所有控制信号（复位有效）
44         pause_k_tb = 0;
45         ff_k_tb = 0;
46         fb_k_tb = 0;
47         rst_k_tb = 0;          // 复位：低电平有效
48         video_choose_tb = 2'b00;
49         // 保持复位 10ns
50         #10 rst_k_tb = 1;      // 释放复位（此后不再拉低！）
51         // 正常播放一段时间
52         #100;
53         // 暂停
54         pause_k_tb = 1;
55         #50;
56         pause_k_tb = 0;        // 继续播放
57         // 快进
58         #50;
59         ff_k_tb = 1;
60         #50;
61         ff_k_tb = 0;
62         // 快退
63         #50;
64         fb_k_tb = 1;
65         #50;
66         fb_k_tb = 0;
67         // 切换视频源
68         #50;
69         video_choose_tb = 2'b01;
70         // 再运行一段时间
71         #200;
72         // 结束仿真
73         $finish;
74     end
75 endmodule
```


同时对原模块进行调整：

将 video_addr_gen.v 改为 video_addr_gen_for_test.v

将 video_player.v 改为 video_player_for_test.v

```
1  module video_player_for_test (
2      input      clk_sys,
3      input      pause_k,
4      input      ff_k,
5      input      fb_k,
6      input      rst_k,
7      input [1:0] video_choose,
8      output [15:0] data_col,
9      output [3:0] curr_col
10 );
11
12 wire [13:0] addr;
13
14 // 实例化桩地址生成器
15 video_addr_gen_for_test u_addr_gen (
16     .clk(clk_sys),
17     .rst_n(rst_k),
18     .pause(pause_k),
19     .ff(ff_k),
20     .fb(fb_k),
21     .video_sel(video_choose),
22     .addr_out(addr),
23     .col_out(curr_col)
24 );
25
26 // 数据输出：仅用于仿真可见，无实际意义
27 assign data_col = {4{curr_col}}; // 重复 curr_col 填充 16 位
28
29 endmodule

```

```
1  module video_addr_gen_for_test (
2      input      clk,
3      input      rst_n,
4      input      pause,
5      input      ff,          // 快进
6      input      fb,          // 快退
7      input [1:0] video_sel,
8      output reg [13:0] addr_out,
9      output reg [3:0] col_out // 假设 curr_col 是 4-bit 列计数器
10 );
11
12 // 简化：只模拟列计数器和简单地址（用于测试控制逻辑）
13 always @(posedge clk or negedge rst_n) begin
14     if (!rst_n) begin
15         col_out <= 0;
16         addr_out <= 0;
17     end else begin
18         if (!pause) begin
19             if (ff)
20                 col_out <= col_out + 2; // 快进：跳两列
21             else if (fb)
22                 col_out <= (col_out >= 2) ? col_out - 2 : 0; // 快退
23             else
24                 col_out <= col_out + 1; // 正常播放
25
26             // 地址简单映射（例如 addr = col * 10）
27             addr_out <= {col_out, 4'd0}; // 左移4位模拟
28         end
29         // 如果 pause=1, 保持不变
30     end
31 end
32
33 endmodule

```

1) 编译文件

命令行: iverilog -o video_player_for_test.out video_player_for_test.v
video_addr_gen_for_test.v video_player_tb.v

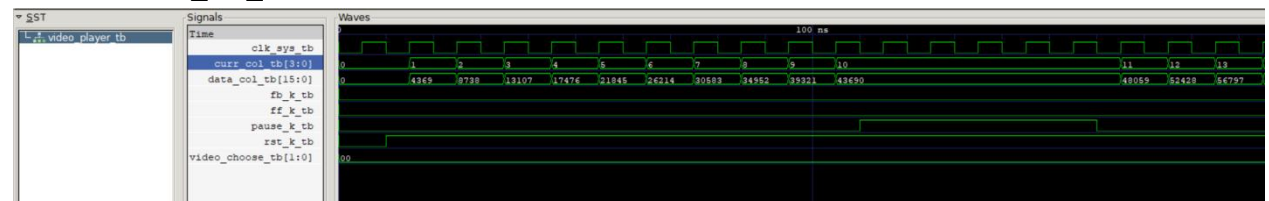
2) 生成 .vcd 波形文件

命令行: vvp video_player_for_test.out

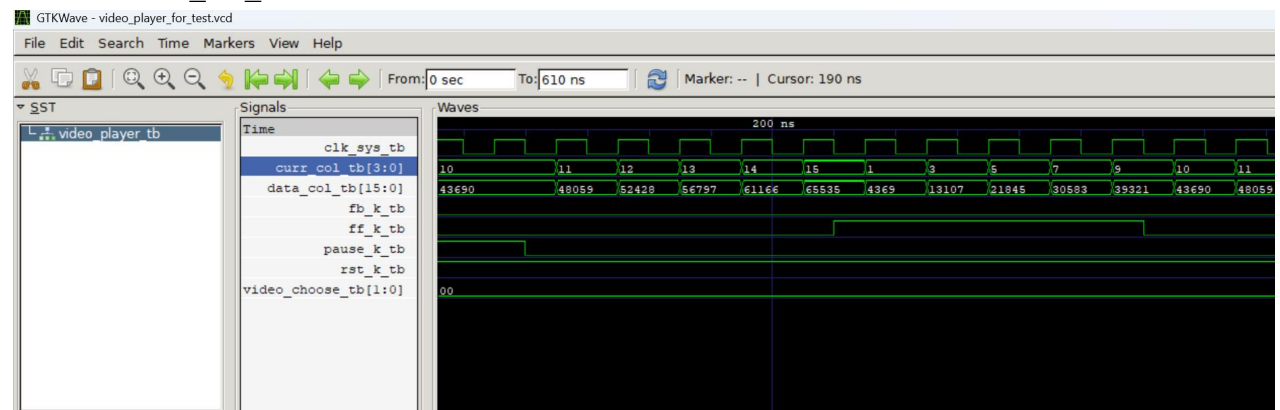
3) 观察波形

命令行: gtkwave video_player_for_test.vcd

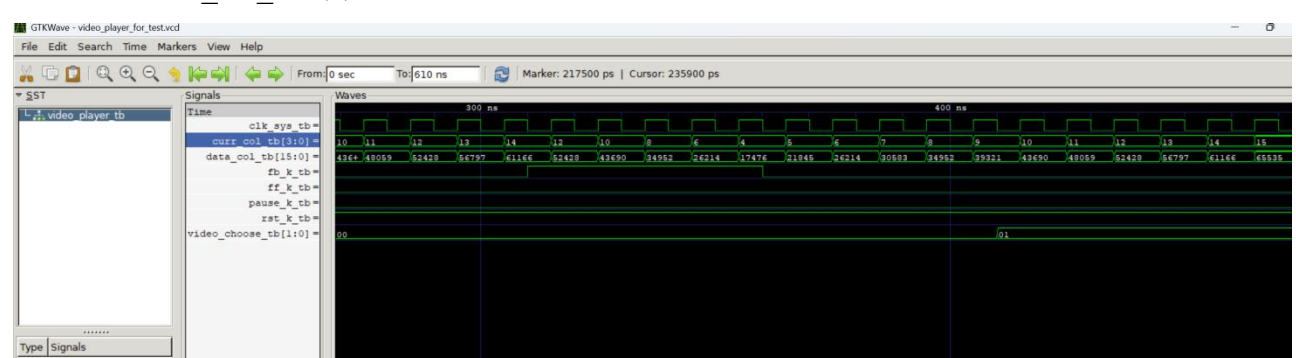
A. 暂停: curr_col_tb 保持不变



B. 快进, curr_col_tb 每次增加 2



C. 快退, curr_col_tb 每次减少 2



2. (额外) debouncing 防抖模块

编写 debouncing_tb.v 代码

```

`timescale 1ns/1ps

module debouncing_tb;

    // -----
    // 时钟产生 (周期10ns = 100MHz)
    // -----
    reg clk_sys = 0;
    always #5 clk_sys = ~clk_sys;

    // -----
    // DUT 输入 / 输出
    // -----
    reg key_b;
    reg resetn;
    wire key;
    wire key_pulse;
    wire [7:0]counter;

    // -----
    // DUT 实例化 (delayT很小方便观察)
    // -----
    debouncing_fsm #(
        .bitwidth(8),
        .delayT(50)//这里修改了位宽和delayT
    ) uut (
        .clk_sys(clk_sys),
        .resetn(resetn),
        .key_b(key_b),
        .key(key),
        .key_pulse(key_pulse),
        .counter(counter)
    );

    // 波形输出
    initial begin
        $dumpfile("debouncing.vcd");
        $dumpvars(0, debouncing_tb);
    end

// 测试流程
initial begin
    $display("=== Debouncing Testbench Start ===");
    // 初始状态
    resetn = 1;
    #50;
    // 复位
    resetn = 0;
    #30;
    resetn = 1;
    #50;

    key_b = 1; // 初始未按下
    #50;

    // 模拟有抖动的按键按下
    // 抖动序列: 1->0->1->0->1->0->保持0
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #100; // 稳定按下 (低电平)

    // 保持按下状态一段时间
    #100;

    // 模拟抖动释放: 0->1->0->1->保持1
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #2;
    key_b = 0; #2;
    key_b = 1; #100;

    // 等待波形稳定
    #200;

    $display("=== Simulation Finished ===");
    $finish;
end

```

1) 编译文件

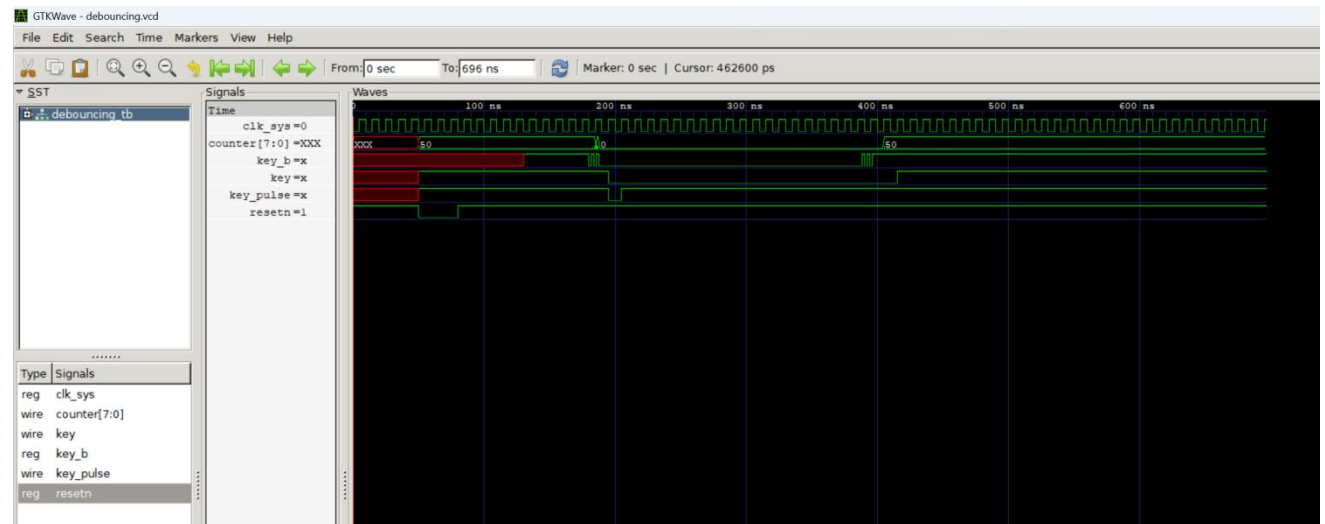
命令行: iverilog -o debouncing.out debouncing.v debouncing_tb.v

2) 生成 .vcd 波形文件

命令行: vvp debouncing.out

3) 观察波形

命令行: gtkwave debouncing.vcd



七、附加实验仿真波形

无附加实验，两个实验均为必做实验。